

Fase 0 — Guía Completa del Intérprete de Pseudocódigo (Ejemplo 17)

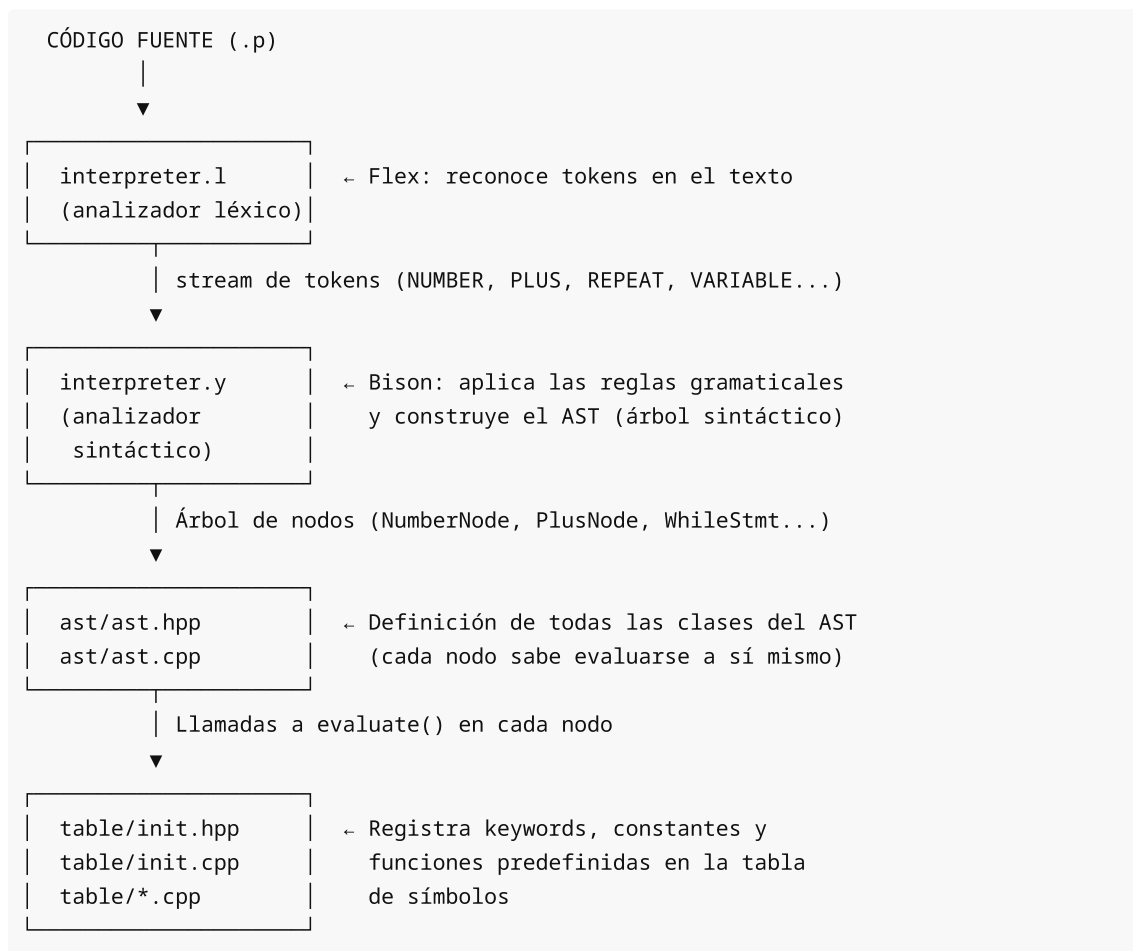
Documento de estudio para el examen de PL. Basado en el Ejemplo 17. Objetivo: entender CÓMO funciona cada archivo y CÓMO se conectan entre sí.

Índice

1. [Arquitectura General](#)
2. [interpreter.l — El analizador léxico](#)
3. [table/init.hpp — La inicialización de la tabla de símbolos](#)
4. [interpreter.y — La gramática](#)
5. [ast/ast.hpp — Las clases del AST](#)
6. [ast/ast.cpp — Implementaciones clave](#)
7. [Las 3 trampas NO obvias del examen](#)
8. [Apéndice: Referencia rápida por pregunta](#)

1. Arquitectura General

El intérprete se compone de **5 archivos fuente** que se conectan en cadena:



Flujo de una sentencia como `print 7 // 3; :`

1. Flex la lee caracter a caracter:

"print" → busca en tabla → es keyword → devuelve PRINT
"7" → coincide con {NUMBER} → devuelve NUMBER
"//" → coincide con "/" → devuelve DIV_ENTERA
"3" → coincide con {NUMBER} → devuelve NUMBER
";" → devuelve SEMICOLON

2. Bison recibe los tokens y aplica reglas:

NUMBER DIV_ENTERA NUMBER → exp DIV_ENTERA exp → IntegerDivisionNode
PRINT IntegerDivisionNode → print → PrintStmt
PrintStmt SEMICOLON → stmt → se añade al stmtlist

3. Se evalúa el árbol:

PrintStmt.evaluate()
→ IntegerDivisionNode.evaluateNumber()
→ left.evaluateNumber() → 7.0
→ right.evaluateNumber() → 3.0
→ (long)7 / (long)3 = 2
→ imprime "print: 2"

Archivos que tocas en el examen:

Archivo	¿Lo modificas?	¿Qué añades?
interpreter.l	✓ SÍ	Líneas de operadores ("//", "++", etc.)
interpreter.y	✓ SÍ	%token, precedencia, reglas gramaticales
table/init.hpp	✓ SÍ (solo D)	Entradas en keyword[]
ast/ast.hpp	✗ NO (ya está completo)	—
ast/ast.cpp	✗ NO (ya está completo)	—

2. interpreter.l — El analizador léxico

Archivo: parser/interpreter.l

Propósito: Leer el código fuente carácter a carácter y agruparlos en **tokens** que Bison pueda entender.

2.1 Estructura del archivo

	%{ ... %}		→ Código C++ de cabecera (includes)		líneas 17-31
	%option ...		→ Opciones de Flex		líneas 33-35
	Definiciones		→ Patrones con nombre (DIGIT, NUMBER...)		líneas 37-55
	%%				
	Reglas		→ patrón { acción }		líneas 57-177

```
%%
```

2.2 Cabecera (líneas 17-31) — No se toca

```
%{
#include "../ast/ast.hpp"           // ← clases del AST
#include "interpreter.tab.h"        // ← tokens definidos en .y (generado por
bison)
#include "../table/table.hpp"       // ← tabla de símbolos
#include "../error/error.hpp"       // ← funciones de error
extern lp::Table table;            // ← la tabla de símbolos (vive en
interpreter.cpp)
extern int lineNumber;             // ← contador de líneas
%}
```

El lexer necesita:

- **interpreter.tab.h** : saber qué números tienen los tokens (PRINT=257, WHILE=258, etc.)
- **table** : poder preguntar si un identificador es keyword o variable
- **ast.hpp** : poder crear `NumericVariable` cuando ve un identificador nuevo

2.3 Opciones (líneas 33-35)

```
%option case-insensitive    → "PRINT", "Print", "print" son iguales
%option noyywrap            → solo un archivo de entrada
```

2.4 Definiciones de patrones (líneas 37-49) — No se toca

```
DIGIT      [0-9]
LETTER     [a-zA-Z]
NUMBER     {DIGIT}+(\.{DIGIT}+)?([eE][+-]?{DIGIT}+)? → 3, 3.14, 2e-5, 1.5E+3
IDENTIFIER {LETTER}(_?({LETTER}|{DIGIT})+)*          → x, var_1, contador, miVar_2
BAD_ID     {LETTER}({LETTER}|{DIGIT}|_)*             → x_, a__b (detecta mal
formados)
STRING_LIT \"([^\"]|\\\".)*\"                          → 'hola', 'a\nb', 'it\\'s'
```

Importante: `NUMBER` acepta enteros (7), reales (3.14) y notación científica (1e-5).
`IDENTIFIER` permite guiones bajos pero **no al final** y **no dobles** (se valida en la regla).

2.5 Reglas (líneas 57-176)

Cada regla tiene el formato:

```
patrón { código C++ }
```

Cuando el patrón coincide, se ejecuta el código. Normalmente asigna un valor a `yylval` y devuelve el token.

a) Reglas de formato (líneas 60-66) — No se toca

[\t]	{ ; }	→ ignorar espacios y tabs
\n	{ lineNumber++; }	→ contar líneas
";"	{ return SEMICOLON; }	→ punto y coma
","	{ return COMMA; }	→ coma

b) Números (líneas 68-71) — No se toca

```
{NUMBER} {
    yylval.number = atof(yytext);    ← convertir "7" → 7.0
    return NUMBER;                  ← decirle a bison que es un NUMBER
}
```

`yylval` es una unión que contiene todos los tipos posibles de valor. La declaración está en `interpreter.y`:

```
%token <number> NUMBER    → yylval.number (double)
%token <string> VARIABLE   → yylval.string (char*)
%token <logic> BOOL        → yylval.logic (bool)
```

c) Comentarios (líneas 73-83) — No se toca

```
#.*      { ; }           → comentario de línea: # todo esto se ignora
"(*"      { BEGIN(COMMENT); } → entrada a comentario de bloque
<COMMENT>"*)" { BEGIN(INITIAL); } → salida de comentario de bloque
<COMMENT>\n { lineNumber++; }
<COMMENT>.\n { ; }
<COMMENT><<EOF>> {          → error: comentario sin cerrar
    warning("Lexical error", "Comment without closing");
    return 0;              → termina el programa
}
```

d) Strings (líneas 85-112) — No se toca

```
{STRING_LITERAL} {
    // Quitar comillas y procesar escapes (\n, \t, \\", \')
    yylval.string = result;    ← el string ya procesado
    return STRING;
}
```

e) Identificadores — LA REGLA CLAVE (líneas 114-137) — No se toca

```
{IDENTIFIER} {
    // Validar que no acabe en _ ni tenga __
    if (yytext[yytextlen-1] == '_') { ... error ... }
    if (strstr(yytext, "__") != NULL) { ... error ... }

    std::string identifier(yytext);
    yylval.string = strdup(yytext);
}
```

```

    if (table.lookupSymbol(identifier) == false) {
        // No está en la tabla → es una variable NUEVA
        lp::NumericVariable *n = new lp::NumericVariable(..., VARIABLE, UNDEFINED,
0.0);
        table.installSymbol(n);
        return VARIABLE;           ← devuelve VARIABLE
    } else {
        // Sí está en la tabla → es keyword, constante o función
        lp::Symbol *s = table.getSymbol(identifier);
        return s->getToken();       ← devuelve su token (PRINT, REPEAT, PI...)
    }
}

```

Esta es la regla que conecta con `init.hpp` .

Cuando el lexer ve `repeat` :

1. Busca `"repeat"` en la tabla de símbolos
2. Si `init.hpp` registró `{"repeat", REPEAT}` → la tabla tiene un `Keyword("repeat", REPEAT)` → devuelve `REPEAT`
3. Si `init.hpp` NO registró `repeat` → la tabla no lo tiene → crea `NumericVariable("repeat")` → devuelve `VARIABLE`

Moraleja: Si olvidas `init.hpp` , el lexer nunca devuelve `REPEAT` . El parser nunca reconoce el bucle.

f) Operadores (líneas 143-164) — LO QUE SÍ TOCAS

```

":="    { return ASSIGNMENT; }    ← asignación
"//"    { return DIV_ENTERA; }    ← división entera (A)
"?"    { return '?'; }          ← ternario (B)
":."    { return ':'; }          ← ternario (B)
"++"    { return INCREMENTO; }   ← incremento (C)
"--"    { return DECREMENTO; }   ← decremento (C)
"+:="   { return MAS_IGUAL; }    ← suma compuesta (C)
"-:="   { return MENOS_IGUAL; }  ← resta compuesta (C)

```

Cada línea sigue el mismo patrón: `"literal" { return TOKEN; } .`

Importante: Flex empareja la regla más larga disponible. Por eso `"//"` debe ir DESPUÉS de `"/"` (si no, `"/"` se emparejaría antes). Y `":="` debe ir ANTES de `":"` por la misma razón.

g) Manejo de errores léxicos (líneas 166-177) — No se toca

```

<<EOF>> { return 0; }          → fin de archivo
.        { BEGIN(ERROR); }     → carácter desconocido → estado ERROR

```

3. table/init.hpp — La inicialización de la tabla de símbolos

Archivo: `table/init.hpp`

Propósito: Definir los arrays que `init.cpp` recorrerá para poblar la tabla de símbolos.

3.1 Estructura

```
// Cada array termina con {"", 0} (centinela)
// init.cpp recorre el array hasta encontrar ese marcador

static struct { string name; double value; } numericConstant[] = { ... }; // PI,
E, ...
static struct { string name; bool value; } logicalConstant[] = { ... }; // true,
false
static struct { string name; int token; } keyword[] = { ... }; //
print, if, repeat...
static struct { string name; function_1; } function_1[] = { ... }; // sin,
cos...
static struct { string name; function_0; } function_0[] = { ... }; //
random
static struct { string name; function_2; } function_2[] = { ... }; // atan2
```

3.2 El array `keyword[]` (líneas 69-104) — LO QUE TOCAS (solo D)

```
static struct {
    std::string name;
    int token;
} keyword[] = {
    {"print", PRINT},
    {"read", READ},
    {"read_string", READ_STRING},
    {"if", IF},
    {"then", THEN},
    {"else", ELSE},
    {"end_if", END_IF},
    {"while", WHILE},
    {"do", DO},
    {"end_while", END_WHILE},
    {"repeat", REPEAT}, // ← D: ¡NO OLVIDAR!
    {"until", UNTIL}, // ← D: ¡NO OLVIDAR!
    {"for", FOR},
    {"end_for", END_FOR},
    {"from", FROM},
    {"step", STEP},
    {"to", TO},
    {"switch", SWITCH},
    {"case", CASE},
    {"default", DEFAULT},
    {"end_switch", END_SWITCH},
    {"clear_screen", CLEAR_SCREEN},
    {"place", PLACE},
    {"or", OR},
    {"and", AND},
    {"not", NOT},
    {"mod", MODULO},
```

```
    {"", 0} // ← centinela (siempre al final)
};
```

¿Qué hace `init.cpp` con esto?

```
for (i=0; keyword[i].name.compare("")!=0; i++) {
    k = new lp::Keyword(keyword[i].name, keyword[i].token);
    t.installSymbol(k); // lo mete en la tabla de símbolos
}
```

El flujo completo para `repeat` :

```
init.hpp: {"repeat", REPEAT}
    ↓ init.cpp lo recorre
    ↓ new Keyword("repeat", REPEAT) → se mete en la tabla
    ↓
interpreter.l: el lexer ve "repeat"
    ↓ table.lookupSymbol("repeat") → true (existe)
    ↓ s->getToken() → devuelve REPEAT
    ↓
interpreter.y: la regla match REPEAT → repeat: REPEAT ...
```

Pregunta D en el examen:

Te dan el código sin `repeat` ni `until` en `init.hpp`. Tú debes:

1. Encontrar el array `keyword[]` en `table/init.hpp` (líneas 69-104)
2. Añadir `{"repeat", REPEAT}`, y `{"until", UNTIL}`,
3. También en `interpreter.y`: añadir `%token REPEAT UNTIL` y `%type ... repeat` y la regla

3.3 Otros arrays — No se tocan en el examen

```
numericConstant[] = { {"PI", 3.1415...}, {"E", 2.718...}, ... };
logicalConstant[] = { {"true", true}, {"false", false} };
function_1[]      = { {"sin", sin}, {"cos", cos}, {"sqrt", sqrt}, ... };
function_0[]      = { {"random", Random} };
function_2[]      = { {"atan2", Atan2} };
```

4. interpreter.y — La gramática

Archivo: `parser/interpreter.y`

Propósito: Definir las reglas gramaticales que Bison usa para analizar los tokens y construir el AST.

4.1 Estructura del archivo

<code>%{ ... %}</code>	→ Código C++ de cabecera	líneas 156-197
<code>%token / %type</code>	→ Declaraciones de tokens y tipos	

Precedencia → %left, %right, %nonassoc	líneas 199-236
%%	
Reglas gramaticales → programa → stmtlist → stmt...	líneas 239-729
%%	

4.2 Declaraciones de tokens (líneas 156-197)

%type <expNode> exp	→ exp devuelve un ExpNode*
%type <st> stmt asgn print read if while for_stmt ...	→ stmt devuelve un Statement*
%type <stmts> stmtlist	→ stmtlist devuelve lista
%type <parameters> listOfExp restOfListOfExp	→ lista de expresiones
%token SEMICOLON COMMA LPAREN RPAREN	→ tokens básicos
%token PRINT READ READ_STRING IF THEN ELSE END_IF	→ keywords
%token WHILE DO END_WHILE REPEAT UNTIL	→ D: REPEAT UNTIL
%token FOR END_FOR FROM STEP TO	→ bucle for
%token SWITCH CASE DEFAULT END_SWITCH	→ switch
%token CLEAR_SCREEN PLACE	→ screen
%token OR AND NOT	→ lógicos
%token DIV_ENTERA CONCAT	→ A: DIV_ENTERA
%token EQUAL NOT_EQUAL MODULO	→ comparación
%token ASSIGNMENT MAS_IGUAL MENOS_IGUAL INCREMENTO DECREMENTO	→ C: MAS_IGUAL...
%token <string> STRING	→ string literal
%token <number> NUMBER	→ número
%token <logic> BOOL	→ booleano
%token <string> VARIABLE UNDEFINED CONSTANT BUILTIN	→ identificadores

Las que puedes tener que tocar:

Pregunta	Tokens a declarar
A	añadir DIV_ENTERA en %token DIV_ENTERA CONCAT (línea 189)
B	'?' y ':' son literales → NO se declaran
C	MAS_IGUAL MENOS_IGUAL INCREMENTO DECREMENTO ya están en línea 191
D	REPEAT UNTIL ya están en línea 179 (pero pueden faltar en init.hpp)

4.3 La tabla de precedencia (líneas 199-236) — CLAVE para 0 conflictos

/* ORDEN: de MENOR a MAYOR precedencia */	
%right '?' ':'	/* B: TERNARIO (la más baja de todas) */
%left OR	/* lógicos */
%left AND	
%nonassoc ... >= <= > <	/* relacionales */
%left NOT	


```

%left PLUS MINUS          /* aritméticos */
%left CONCAT              /* concatenación || */
%left MULTIPLICATION DIVISION MODULO DIV_ENTERA /* A: DIV_ENTERA aquí */
%left LPAREN RPAREN

%right INCREMENTO DECREMENTO /* C: ++/-- */
%nonassoc UNARY             /* operadores unarios */

%right POWER               /* potencia (la más alta) */

%nonassoc THEN ELSE        /* resuelve el conflicto if-else */

```

Regla de oro: Las líneas más abajo = mayor precedencia.

POWER es lo que más prio tiene. ?: es lo que menos.

¿Qué hace cada declaración?

Declaración	Significado
%left	Asociatividad por la izquierda: $a+b+c = (a+b)+c$
%right	Asociatividad por la derecha: $a^b^c = a^b^c$
%nonassoc	No asociativo: $a<b<c$ es error

¿Por qué %right '?' ':' va el primero (menor prioridad)?

Porque queremos que $1<2 ? 10 : 20$ se parse como $(1<2) ? (10) : (20)$.

El $<$ tiene más prioridad que $?:$, así que $<$ se evalúa primero y el $?:$ luego.

Si %right '?' ':' estuviera después de los relacionales (como estaba originalmente), $?:$ tendría MÁS prioridad que $<$, y $1<2?10:20$ se parsearía como $1 < (2?10:20)$ → **resultado incorrecto**.

4.4 Reglas gramaticales

a) Programa (líneas 244-255)

```

program: stmtlist
      { $$ = new lp::AST($1); root = $$; }

```

El programa es una lista de sentencias. Se crea un objeto AST con la lista.

b) Lista de sentencias (líneas 257-294)

```

stmtlist: /* vacío */                                → epsilon: lista vacía
      { $$ = new std::list<lp::Statement*>(); }

      | stmtlist stmt                                → lista + nueva sentencia
      { $$ = $1; $$->push_back($2); }

```

stmtlist error	→ recuperación de errores
{ \$\$ = \$1; yyclearin; }	

En modo interactivo, después de cada `;` se evalúa inmediatamente la lista acumulada (líneas 270-283).

c) Sentencias (líneas 297-349)

stmt: SEMICOLON	→ sentencia vacía	
asgn SEMICOLON	→ asignación	
print SEMICOLON	→ print	
read SEMICOLON	→ read	
if	→ if-then-else	
while	→ while-do	
for_stmt	→ for	
repeat	→ repeat-until	← D
do_while	→ do-while	
clear_screen	→ cls	
place	→ place	
switch_stmt	→ switch	

Pregunta D: Si falta `| repeat` aquí, el parser nunca reconoce un bucle repeat, aunque la regla `repeat:` esté definida.

d) Asignaciones (líneas 455-505) — LO QUE TOCAS (C)

asgn: VARIABLE ASSIGNMENT exp	→ x := 5	
VARIABLE ASSIGNMENT asgn	→ x := y := 5 (asignación múltiple)	
INCREMENTO VARIABLE	→ ++x	← C
DECREMENTO VARIABLE	→ --x	← C
VARIABLE INCREMENTO	→ x++	← C
VARIABLE DECREMENTO	→ x--	← C
VARIABLE MAS_IGUAL exp	→ x += 3	← C
VARIABLE MENOS_IGUAL exp	→ x -= 2	← C
CONSTANT ASSIGNMENT exp	→ error: constante	
CONSTANT ASSIGNMENT asgn	→ error: constante	

Las 6 reglas de C (líneas 467-493) crean los nodos del AST correspondientes.

e) Print y Read (líneas 507-529)

```
print: PRINT exp
{ $$ = new lp::PrintStmt($2); }

read: READ LPAREN VARIABLE RPAREN
{ $$ = new lp::ReadStmt($3); }
| READ LPAREN CONSTANT RPAREN
{ execerror("no se puede modificar constante"); }
| READ_STRING LPAREN VARIABLE RPAREN
{ $$ = new lp::ReadStmt($3); }
```

¿Cómo sabe `PrintStmt` qué imprimir?

Llama a `_exp->getType()` para saber si es `NUMBER`, `STRING` o `BOOL`, y luego llama al `evaluate*()` correspondiente.

f) Expresiones (líneas 532-630) — LO QUE TOCAS (A y B)

<code>exp: NUMBER</code>	→ 7, 3.14, 2e-5	
<code>STRING</code>	→ 'hola'	
<code>BOOL</code>	→ true, false	
<code>exp PLUS exp</code>	→ suma	
<code>exp MINUS exp</code>	→ resta	
<code>exp MULTIPLICATION exp</code>	→ multiplicación	
<code>exp DIVISION exp</code>	→ división	
<code>exp DIV_ENTERA exp</code>	→ división entera	← A
<code>LPAREN exp RPAREN</code>	→ (exp)	
<code>PLUS exp %prec UNARY</code>	→ +expr (unario)	
<code>MINUS exp %prec UNARY</code>	→ -expr (unario)	
<code>exp MODULO exp</code>	→ resto	
<code>exp '?' exp ':' exp %prec '?'</code>	→ ternario	← B
<code>exp POWER exp</code>	→ potencia	
<code>VARIABLE</code>	→ variable	
<code>CONSTANT</code>	→ constante	
<code>BUILTIN LPAREN listOfExp RPAREN</code>	→ función	
<code>exp GREATER_THAN exp</code>	→ relacionales	
... (más relacionales)		
<code>exp AND exp</code>	→ lógicos	
<code>exp OR exp</code>		
<code>NOT exp</code>		
<code>exp CONCAT exp</code>	→ concatenación	

¿Qué hace `%prec UNARY` ?

Le dice a Bison: "trata esta regla como si tuviera la precedencia de `UNARY`". Así `-3+2` se parsea como `(-3)+2`, no `-(3+2)`.

¿Qué hace `%prec '?'` ?

Igual: "trata el ternario como si tuviera la precedencia de `'?'`". Sin esto, el ternario generaría conflictos porque `:` también aparece en `switch`.

g) Bucle `repeat-until` (líneas 393-399) — LO QUE TOCAS (D)

```
repeat: REPEAT controlSymbol stmtlist UNTIL exp SEMICOLON
{
    $$ = new lp::RepeatStmt(new lp::BlockStmt($3), $5);
    control--;
}
```

¿Qué es `controlSymbol` ?

Una regla `epsilon` (vacía) que solo incrementa un contador:

```
controlSymbol: /* epsilon */
{ control++; }
```

Este contador `control` evita que en modo interactivo se ejecuten bloques incompletos (como un `while` sin su `end_while`). Cuando `control > 0`, el modo interactivo no ejecuta hasta que el bloque esté completo.

¿Por qué `do_while` necesita DOS `controlSymbol` ? (línea 402)

```
do_while: DO controlSymbol stmtlist WHILE controlSymbol exp SEMICOLON
```

El primer `controlSymbol` (tras `DO`) es para el `do`.

El segundo (tras `WHILE`) es para **resolver el conflicto** con `while: WHILE controlSymbol ...`.

Sin el segundo `controlSymbol`, cuando el parser ve `WHILE` no sabe si está en un `while` (esperando `controlSymbol`) o en un `do-while` (esperando `exp`). Eso genera **9 shift/reduce conflicts**.

Con el segundo `controlSymbol`, ambos caminos empiezan igual (`WHILE → controlSymbol`) y no hay conflicto.

La regla de `do_while` CORREGIDA (0 conflictos):

```
do_while: DO controlSymbol stmtlist WHILE controlSymbol exp SEMICOLON
{
    $$ = new lp::DoWhileStmt(new lp::BlockStmt($3), $6);
    control--;           // por el primer controlSymbol (DO)
    control--;           // por el segundo controlSymbol (WHILE)
}
```

4.5 ¿Cómo se numeran \$1, \$2, \$3?

Cada símbolo en la parte derecha de una regla tiene un número:

```
repeat: REPEAT controlSymbol stmtlist UNTIL exp SEMICOLON
        $1      $2          $3      $4  $5  $6
```

```
do_while: DO controlSymbol stmtlist WHILE controlSymbol exp SEMICOLON
          $1      $2          $3      $4      $5      $6  $7
```

Errores comunes:

- Usar `$5` en vez de `$6` en `do_while` después de añadir el segundo `controlSymbol`
- Olvidar que `$2` es `controlSymbol` (epsilon), no tiene valor pero ejecuta código

5. ast/ast.hpp — Las clases del AST

No se toca en el examen, pero debes entender la jerarquía para saber qué nodos se crean en las reglas de `.y`.

5.1 Jerarquía de Expresiones

```
ExpNode (clase base abstracta)
|
| virtual int getType()
| virtual double evaluateNumber() → por defecto devuelve 0.0
| virtual bool evaluateBool() → por defecto devuelve false
| virtual std::string evaluateString() → por defecto devuelve ""
|
├── NumberNode
|   | getType() = NUMBER
|   | evaluateNumber() → devuelve _number
|   |
├── StringNode
|   | getType() = STRING
|   | evaluateString() → devuelve _value
|   |
├── VariableNode
|   | getType() → consulta la tabla de símbolos
|   | evaluateNumber() → ((NumericVariable*)var)->getValue()
|   |
├── ConstantNode
|   | igual que VariableNode pero para constantes (PI, E, true, false)
|   |
├── OperatorNode (abstracta)
|   | _left, _right (dos hijos)
|   |
|   ├── NumericOperatorNode
|   |   | getType() = NUMBER
|   |   | ├── PlusNode, MinusNode, MultiplicationNode, DivisionNode
|   |   | ├── IntegerDivisionNode ← A
|   |   | ├── ModuloNode, PowerNode
|   |   | └── ConcatenationNode
|   |
|   ├── ComparisonNode
|   |   | getType() = BOOL
|   |   | ├── GreaterThanNode, LessThanNode, EqualNode...
|   |   |
|   ├── LogicalNode
|   |   | getType() = BOOL
|   |   | ├── AndNode, OrNode
|   |   |
|   └── NumericUnaryOperatorNode / LogicalUnaryOperatorNode
|       | getType() según el operando
|       | ├── UnaryMinusNode, UnaryPlusNode
|       | └── NotNode
|       |
└── TernaryNode ← B
    | getType() = _true->getType()
    | evaluateNumber() = _cond->evaluateBool() ? _true->evaluateNumber() : _false-
    >evaluateNumber()
    | evaluateBool() = _cond->evaluateBool() ? _true->evaluateBool() : _false-
```

```
>evaluateBool()
    evaluateString() = _cond->evaluateBool() ? _true->evaluateString() : _false-
>evaluateString()
```

5.2 Jerarquía de Sentencias

```
Statement (clase base abstracta)
|   virtual void evaluate()
|
|— EmptyStmt
|— PrintStmt
|— ReadStmt
|— AssignmentStmt
|— IfStmt
|— WhileStmt
|— DoWhileStmt
|— RepeatStmt           ← D
|— ForStmt
|— SwitchStmt
|— BlockStmt
|— PreIncrementStmt     ← C
|— PostIncrementStmt    ← C
|— CompoundAssignmentStmt ← C
|— ClearScreenStmt
|— PlaceStmt
```

5.3 Cómo evalúa PrintStmt

```
void lp::PrintStmt::evaluate() {
    switch (this->_exp->getType()) {
        case NUMBER:
            cout << this->_exp->evaluateNumber() << endl;
            break;
        case STRING:
            cout << this->_exp->evaluateString() << endl;
            break;
        case BOOL:
            if (this->_exp->evaluateBool())
                cout << "true" << endl;
            else
                cout << "false" << endl;
            break;
    }
}
```

Esto es lo que hace que `getType()` sea crítico.

Si `TernaryNode::getType()` devuelve `_true->getType()` y `_true` es un `NumberNode(10)`, entonces `getType` devuelve `NUMBER` y se llama a `evaluateNumber()`. **Funciona automáticamente.**

6. ast/ast.cpp — Implementaciones clave

No se toca en el examen, pero debes entender cómo se evalúa cada nodo.

6.1 Evaluación de IntegerDivisionNode (A)

```
double lp::IntegerDivisionNode::evaluateNumber() {
    double left  = this->_left->evaluateNumber();    // 7.0
    double right = this->_right->evaluateNumber();    // 3.0
    if (std::abs(right) < 1.0e-6)
        execerror("Runtime error: division by zero", "");
    return (double)((long)left / (long)right);        // (long)7/(long)3 = 2 →
    (double)2
}
```

6.2 Evaluación de RepeatStmt (D)

```
void lp::RepeatStmt::evaluate() {
    int maxIter = 100000;
    int iter = 0;
    do {
        if (++iter > maxIter) {
            execerror("Runtime error", "Infinite loop detected in repeat");
            break;
        }
        this->_body->evaluate();
    } while (!this->_cond->evaluateBool());    // ← el ! invierte la lógica
}
```

Diferencia entre repeat-until y do-while:

Bucle	Se ejecuta mientras...
do { body } while(cond)	La condición es true
repeat body until cond	La condición es false (invertida)

6.3 Evaluación de PreIncrementStmt (C)

```
void lp::PreIncrementStmt::evaluate() {
    // 1. Buscar la variable en la tabla
    lp::Variable *var = (lp::Variable *) table.getSymbol(this->_var);
    if (!var) {
        // Si no existe, crearla con valor 0
        var = new lp::NumericVariable(this->_var, VARIABLE, NUMBER, 0.0);
        table.installSymbol(var);
    } else if (var->getType() != NUMBER) {
        var->setType(NUMBER);
    }
}
```

```

// 2. Calcular nuevo valor
double delta = this->_inc ? 1.0 : -1.0;
double newVal = ((lp::NumericVariable*)var)->getValue() + delta;

// 3. Guardar
((lp::NumericVariable*)var)->setValue(newVal);
}

```

El casteo a `NumericVariable*` es necesario porque:

```

Symbol (clase base)
├─ Variable (tiene getName(), getToken(), getType()...)
│   └─ NumericVariable (tiene getValue(), setValue(double))
│       └─ LogicalVariable (tiene getValue(), setValue(bool))
│           └─ StringVariable (tiene getValue(), setValue(string))

```

El método `getValue()` solo existe en `NumericVariable`, no en `Variable`.
 Por eso hay que bajar en la jerarquía: `((NumericVariable*)var)->getValue()`.

7. Las 3 trampas NO obvias del examen

Son los 3 errores que no se descubren por intuición, hay que saberlos.

Trampa 1: Precedencia de `%right '?' ':'` (B)

✗ MAL (genera resultados incorrectos):
`%left PLUS MINUS`
`%nonassoc ... <= >= < >`
`%right '?' ':'` ← aquí NO: da más prioridad a `?:` que a relacionales

✓ BIEN (resultados correctos):
`%right '?' ':'` ← aquí SÍ: `?:` tiene la menor prioridad
`%left OR AND`
`%nonassoc ... <= >= < >`
`%left PLUS MINUS`

Qué hace: `1 < 2 ? 10 : 20` → si `?:` tiene más prioridad que `<`, se parsea como `1 < (2 ? 10 : 20) = 1 < 20 = true`.

Si `?:` tiene menos prioridad que `<`, se parsea como `(1 < 2) ? 10 : 20 = true ? 10 : 20 = 10`.

La segunda es la correcta.

Trampa 2: `controlSymbol` extra en `do_while` (0 sr conflicts)

✗ MAL (9 conflictos):
`do_while: DO controlSymbol stmtlist WHILE exp SEMICOLON`

✓ BIEN (0 conflictos):
`do_while: DO controlSymbol stmtlist WHILE controlSymbol exp SEMICOLON`

Qué hace: El parser ve `WHILE` y no sabe si viene de un `while` (esperando `controlSymbol`) o de un `do_while` (esperando `exp`).

Con `controlSymbol` en ambos sitios, tras `WHILE` siempre reduce `controlSymbol` → no hay ambigüedad.

Además hay que ajustar la acción:

`$5` pasa a ser `$6` y hay que hacer `control--` dos veces.

Trampa 3: Keywords en `init.hpp` además de `%token` (D)

✖

MAL (repeat no funciona):
%token ... REPEAT UNTIL ← en .y sí está
pero NO en init.hpp ← aquí falla

✔

BIEN (repeat funciona):
%token ... REPEAT UNTIL ← en .y
+ {"repeat", REPEAT}, ← en init.hpp (línea 84)
 {"until", UNTIL}, ← en init.hpp (línea 85)

Qué hace: El lexer (línea 128-136) busca el identificador en la tabla de símbolos. Si `repeat` no está registrado como keyword, se crea como variable y devuelve `VARIABLE`. El parser espera `REPEAT` y falla.

8. Apéndice: Referencia rápida por pregunta

Pregunta A — División Entera `//`

Archivo	Qué añadir	Línea
.l	<code>"/" { return DIV_ENTERA; }</code>	160
.y	<code>%token DIV_ENTERA CONCAT (añadir DIV_ENTERA)</code>	189
.y	<code>%left ... MODULO DIV_ENTERA (añadir al final)</code>	222
.y	Regla: <code>exp DIV_ENTERA exp { \$\$ = new lp::IntegerDivisionNode(\$1, \$3); }</code>	563-567

Pregunta B — Ternario `?:`

Archivo	Qué añadir	Línea
.l	<code>"?" { return '?'; }</code>	161
.l	<code>":" { return ':'; }</code>	162
.y	<code>%right '?' ':' (ANTES de %left OR)</code>	204
.y	Regla: <code>exp '?' exp ':' exp %prec '?' { \$\$ = new lp::TernaryNode(\$1, \$3, \$5); }</code>	588-591

Pregunta C — Incremento/Decremento/Compuestos

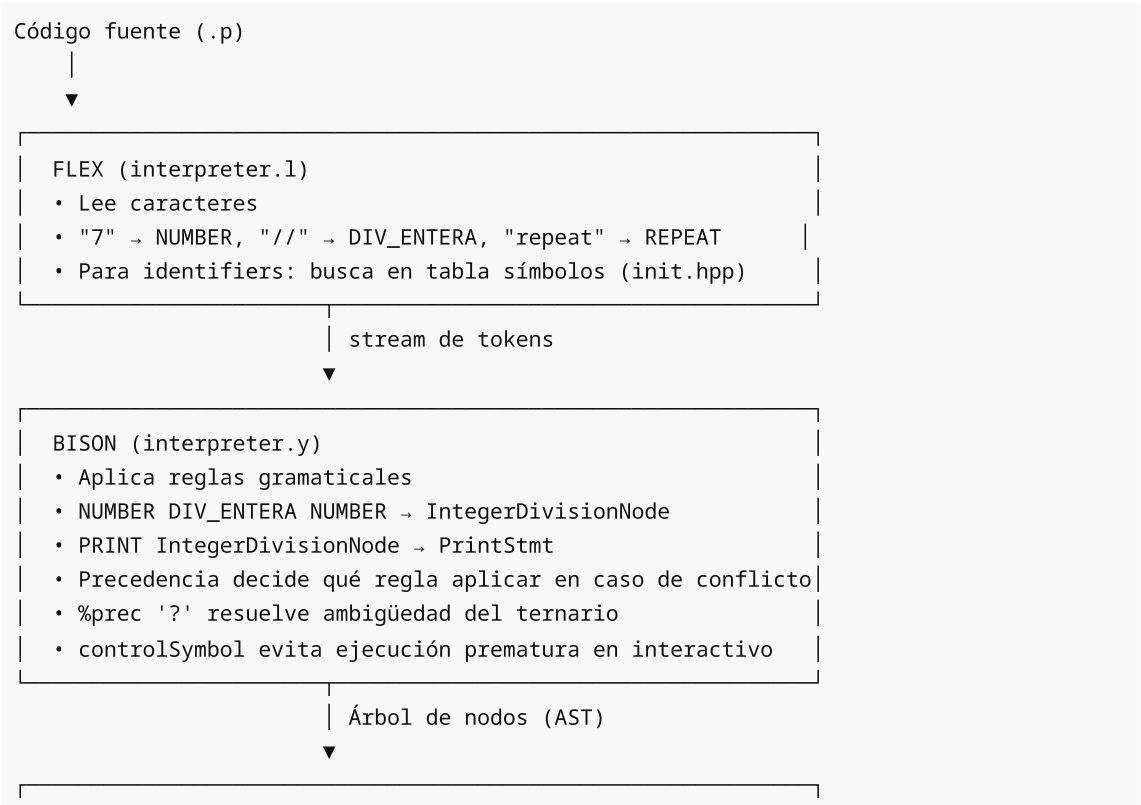
Archivo	Qué añadir	Línea
.l	<code>"++" { return INCREMENTO; }</code>	146

.l	"--" { return DECREMENTO; }	147
.l	"+=" { return MAS_IGUAL; }	143
.l	"-:=" { return MENOS_IGUAL; }	144
.y	%token ... MAS_IGUAL MENOS_IGUAL INCREMENTO DECREMENTO	191
.y	%right INCREMENTO DECREMENTO	226
.y	6 reglas en asgn (++X, --X, X++, X--, X+=e, X-=e)	467-493

Pregunta D — Bucle Repeat-Until

Archivo	Qué añadir	Línea
.y	%token ... REPEAT UNTIL (en línea existente)	179
.y	%type ... repeat (en línea existente)	164
.y	repeat en stmt	330
.y	Regla repeat: REPEAT controlSymbol stmtlist UNTIL exp SEMICOLON { ... }	394-399
init.hpp	{"repeat", REPEAT},	84
init.hpp	{"until", UNTIL},	85

Resumen visual: el flujo completo



AST (ast/ast.hpp + ast/ast.cpp)

- Cada nodo sabe evaluarse a sí mismo
- PrintStmt → `_exp->getType()` → NUMBER? → `evaluateNumber()`
- TernaryNode → `_cond->evaluateBool()` → rama true/false
- RepeatStmt → `do { body } while(!cond)`